



Fundamentos de Internet das Coisas

UNIDADE 01

Fundamentos de sistemas digitais: representações de dados e aritmética de computadores

Olá, estudante! Nesta semana, iremos estudar sobre sistemas de numeração: aritmética binária. Para isso, você acessará os conteúdos selecionados e fará uma avaliação diagnóstica. As atividades instigarão você a reconhecer os sistemas de numeração e converter números de um sistema de numeração para outro. Assim, ao término da semana, você irá interpretar sistemas de numeração em computação. Afinal, por que precisamos conhecer as operações lógicas e aritméticas?

Os computadores, dispositivos IoT, atuadores, transdutores e redes de computadores foram construídos e operam sob premissas digitais. Por sua vez, as premissas digitais utilizam as operações lógicas matemáticas e a aritmética binária. Conhecer o seu funcionamento elementar nos permite compreender, exercitar e implementar alguns dos conceitos e técnicas derivados de antigas e novas tecnologias computacionais disponíveis. Vamos lá?

| Sistemas de numeração

Sistemas de numeração não posicional

Hoje, em uma mercearia, padaria ou supermercado, o sistema de numeração é posicional. Tempos atrás, algumas civilizações antigas utilizavam um sistema de numeração dito não posicional. O exemplo mais fácil é o sistema de numeração romano, um método para representar números.

O sistema de numeração romano é constituído de um conjunto N, com sete algarismos diferentes, cada um representando um valor fixo, independentemente de sua posição relativa no número.

$N = \{I, V, X, L, C, D, M\}$, indicando os seguintes valores: 1, 5, 10, 50, 100, 500 e 1000, respectivamente.

Nesse sistema, não há um símbolo representativo para o zero e os números são definidos da esquerda para a direita. Seus valores são obtidos da seguinte forma:

- a. cada algarismo colocado à direita de um maior é adicionado a este;
- b. cada algarismo colocado à esquerda de outro maior tem seu valor subtraído do maior.

Um sistema não posicional não serve para efetuar cálculos matemáticos, devido às dificuldades para tal. Sua maior utilização é registrar informações numéricas.

Sistema de numeração posicional

Em um sistema posicional de numeração, cada algarismo que forma o número possui um peso, conforme a sua posição, ou seja, o valor é modificado de acordo com sua posição no número. Esse peso cresce da direita para a esquerda.

No sistema decimal, o número 3.733 é constituído de quatro algarismos, sendo que três deles possuem o mesmo valor absoluto (três). No entanto, cada um dos algarismos possui valores diferentes conforme a sua posição no número.

$$3.733_{10} = 3.000 + 700 + 30 + 3$$

$$2.000 = 2 \times 10^3$$

$$700 = 7 \times 10^2$$

$$30 = 3 \times 10^1$$

$$3 = 3 \times 10^0$$

Pelos exemplos dados, foi possível mostrar a ideia de um sistema posicional, neste caso, um número no sistema decimal de numeração, que utilizou potências de 10, por isso decimal. Temos, então, nos sistemas de numeração posicional, o conceito de base, no caso, base 10 (decimal).

Base

O conceito de base de numeração vem da ideia de agrupamento de valores, que permite operações e contagens aritméticas para qualquer valor, de tamanhos diferentes e empregando uma quantidade de símbolos para isso. O motivo de trabalharmos com outras bases vem da necessidade do homem de querer escrever (ou dizer) números elevados (grandezas elevadas), usando um mínimo de símbolos.

Podemos definir a base de um sistema de numeração como sendo a quantidade de símbolos, dígitos ou algarismos diferentes que o referido sistema emprega para representar números.

O sistema decimal utiliza dez símbolos, portanto a sua base é 10. Os símbolos seriam: 0, 1, 2, 3, 4, 5, 6, 7, 8, 9.

O sistema binário (base 2) possui dois símbolos, que seriam: 0 e 1.

O sistema octal (base 8) possui oito símbolos, que seriam: 0, 1, 2, 3, 4, 5, 6, 7.

O sistema hexadecimal (base 16) possui 16 símbolos; como não conhecemos mais do que dez algarismos (dígitos da base 10, que é o nosso sistema de numeração utilizado no dia a dia), utilizamos letras para completar os 16 símbolos, que seriam: 0, 1, 2, 3, 4, 5, 6, 7, 8, 9, A, B, C, D, E, F.

Algarismos e números

Dada uma base B, temos, para essa base, um conjunto de S algarismos.

$$S = \{d_{b-1}, d_{b-2}, d_{b-3}, \dots, d_1, d_0\}$$

Cada número que se deseja escrever será representado por:

$$N = d_{n-1}, d_{n-2}, d_{n-3}, \dots, d_1, d_0$$

Exemplo: 375_{10} , sendo $n = 3$, $n - 1 = 2$; utilizando as representações, temos: $d_2 = 3, d_1 = 7, d_0 = 5$.

Devido à notação posicional, o número $375_{10} = 3 \times 10^2 + 7 \times 10^1 + 5 \times 10^0$ é diferente do número $572_{10} = 5 \times 10^2 + 7 \times 10^1 + 2 \times 10^0$. Sendo posicional e decimal (base 10), cada número pode ser genericamente expresso por: $N = d_{n-1} \times 10^{n-1} + d_{n-2} \times 10^{n-2} + \dots + d_1 \times 10^1 + d_0 \times 10^0$.

Levando em conta que também representamos números fracionários, temos as seguintes fórmulas mais abrangentes:

$$N = d_{n-1} \times 10^{n-1} + d_{n-2} \times 10^{n-2} + \dots + d_0 \times 10^0 + \dots + d_{-1} \times 10^{-1} + d_{-2} \times 10^{-2} + \dots + d_{-m} \times 10^{-m}$$

Exemplos com números fracionários:

$$27,3 = 2 \times 10^1 + 7 \times 10^0 + 3 \times 10^{-1}$$

$$32,12 = 3 \times 10^1 + 2 \times 10^0 + 1 \times 10^{-1} + 2 \times 10^{-2}$$

De forma mais geral, podemos ter a representação de um número para qualquer base:

$$N = \sum_{i=n-1}^{-m} \text{digito}_i \times \text{Base}^i$$

Em que n é o número de dígitos (algarismos) à esquerda da vírgula (parte inteira) e m é o número de dígitos à direita da vírgula (parte fracionária).

Outras bases de numeração

Quando, em um texto, estivermos manipulando números com bases de numerações diferentes, será preciso especificar a base, como podemos ver nos exemplos que seguem:

$$1000101_2, 1000101_b, (1000101)_2 \text{ ou } (1000101)_b$$

$$121_5 \text{ ou } (121)_5$$

$$331_{10}, 331_{d1}, (331)_{10} \text{ ou } (331)_d$$

$$451_7 \text{ ou } (451)_7$$

$$1000101_8 \text{ ou } (1000101)_8$$

$1AF2_{16}$, $1AF2_h$, $(1AF2)_{16}$ ou $(1AF2)_h$

Note que as bases 2, 8 (octal), 10 (decimal) e 16 (hexadecimal) são mais utilizadas, principalmente nas ciências da computação, por isso temos mais formas de representá-las.

| Conversão de bases

Uma vez entendido como representar em notação posicional e como essa notação é aplicável a qualquer base inteira, podemos exercitar a conversão de números de uma base para outra.

Conversão de números de uma base B para a base 10

Para converter um número de uma base qualquer para a base 10 (decimal), basta contar o número de algarismos, a base original e montar o polinômio, obtendo, assim, o número na base 10. Percebemos que o número de dígitos (algarismos) do número é o mesmo que o número de produtos do polinômio.

- 1010001_2 para base 10 (decimal)

Base de origem: base 2

Número de algarismos: 7

$$N = 1 \times 2^6 + 0 \times 2^5 + 1 \times 2^4 + 0 \times 2^3 + 0 \times 2^2 + 0 \times 2^1 + 1 \times 2^0$$

$$N = 64 + 0 + 16 + 0 + 0 + 0 + 1 = 81_{10}$$

- $2B5_{16}$ para base 10 (decimal)

Base de origem: base 16

Número de algarismos: 3

$$N = 2 \times 16^2 + 11 \times 16^1 + 5 \times 16^0$$

$$N = 512 + 176 + 5 = 693_{10}$$

- 1203_7 para base 10 (decimal)

Base de origem: base 7

Número de algarismos: 4

$$N = 1 \times 7^3 + 2 \times 7^2 + 0 \times 7^1 + 3 \times 7^0$$

$$N = 343 + 98 + 0 + 3 = 444_{10}$$



EXERCÍCIO

Exercícios propostos

Converta os seguintes valores de uma base qualquer para a base 10 (decimal).

a) 101001110_2

b) 1110_3

c) $1A_{16}$

d) 1012_8

e) 101003_7



Resolução do exercício



a) 334

- b) 39
 - c) 26
 - d) 522
 - e) 17.153
-

A conversão de números de uma base b para a base 10 é um conceito importante, não apenas na matemática, especialmente na área de computação. Esta habilidade permite que se convertam números de diferentes bases em um formato mais compreensível para os computadores. Existem várias técnicas para converter um número de uma base b para a base 10, incluindo a utilização da representação posicional de cada dígito e a aplicação de uma fórmula matemática específica. Veja na videoaula a seguir.

Conversão de números de uma base B para a base 10

PUCPR Online



Assista no

Conversão de números decimais para uma base B

A conversão de um número de base 10 para outra base qualquer se faz pelo método de divisões sucessivas pela base desejada. Os algoritmos para realizar tal tarefa são:

Algoritmo 1

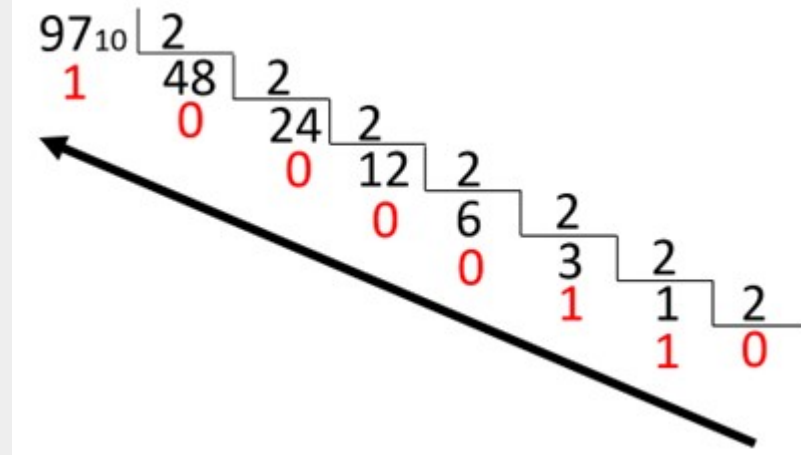
- Enquanto quociente for diferente de zero.
- Dividir dividendo por divisor.
- Extrair resto como algarismo e colocá-lo à esquerda do anterior.
- Repetir até quociente ser igual a zero.

Algoritmo 2

- Enquanto dividendo for maior que o divisor.
- Extrair resto como algarismo e colocá-lo à esquerda do anterior.
- Repetir até divisor ser maior que dividendo.

Exemplos

$$97_{10} = 01100001_2$$



Para o número equivalente na base 2, pegamos o último quociente e o último resto calculado até o primeiro. Assim, temos: 01100001_2 . O último quociente calculado é o algarismo mais significativo (mais à esquerda do número) e o primeiro resto calculado é o número menos significativo (mais à direita do número).

$$2754_{10} = AC2_{16}$$

#ParaTodosVerem 

$$\begin{array}{r}
 2754_{10} \big| 16 \\
 \underline{2} 172 16 \\
 \underline{12} 10 16 \\
 \underline{10} 0
 \end{array}$$

A=10
B=11
C=12
D=13
E=14
F=15

Embora a representação conceitual seja lógica, nem sempre é clara; por esse motivo, assista à videoaula a seguir. Ela trará subsídios dos exemplos apresentados para os sistemas digitais e que trabalham com representações numéricas diferentes da base decimal, como binário e hexadecimal. A reflexão sobre os mecanismos empregados no sistema de numeração comumente usado permitirá uma maior compreensão sobre os sistemas de numeração utilizados nos computadores.

Conversão de números de base 10

PUCPR Online



Assista no



EXERCÍCIO

Exercícios propostos

Converta os seguintes valores decimais em valores binários equivalentes (da base 10 para a base 2).

- a) 126
- b) 127
- c) 128
- d) 254
- e) 255



Resolução do exercício



- a) 01111110
 - b) 01111111
 - c) 10000000
 - d) 11111110
 - e) 11111111
-



EXERCÍCIO

Exercícios propostos

Complete o quadro a seguir:

Base 2	Base 10
	0
	1
	2
	3
	4
	5

	6
	7
	8
	9
	10



Resolução do exercício



1. 0: 0000
2. 1: 0001
3. 2: 0010
4. 3: 0011
5. 4: 0100
6. 5: 0101
7. 6: 0110
8. 7: 0111
9. 8: 1000
10. 9: 1001
11. 10: 1010

Aritmética binária

Tanto a soma quanto a subtração de números binários são semelhantes às que fazemos para o sistema de base 10 (decimal). O que difere é que possuímos apenas dois algarismos (dígitos), que são: 0 e 1.

Soma binária

$$0 + 0 = 0$$

$$0 + 1 = 1$$

$$1 + 0 = 1$$

$$1 + 1 = 10 \text{ (zero e vai um)}$$

Exemplos de operação aritmética de soma com valores em binários inteiros positivos:

#ParaTodosVerem 

$$\begin{array}{r} 111111 \\ 101111 \rightarrow 47_{10} \\ + 010101 \rightarrow 21_{10} \\ \hline 1000100 \rightarrow 68_{10} \end{array}$$

$$\begin{array}{r} 111111 \\ 10110101 \rightarrow 181_{10} \\ + 10001101 \rightarrow 141_{10} \\ \hline 101000010 \rightarrow 322_{10} \end{array}$$



Assista no

| Representação dos dados

O computador funciona por meio da execução sistemática de instruções que orientam a realização de algum tipo de operação sobre valores, os quais são entendidos como dados.

Para representar esses dados, surgiu a necessidade de agrupamentos de bits, visto que 1 bit não significa muita coisa. Com o desenvolvimento da indústria da computação, adotou-se que o menor grupo de bits representativo seria o byte – a IBM foi a primeira a utilizar e ficou como regra para os outros fabricantes –; 1 byte é um conjunto de 8 bits.

Outro grupo de bits, bastante utilizado, é a palavra, que é o tamanho do dado que circula dentro do computador (entre microprocessador, memória principal, memória secundária etc.). Uma palavra define o tamanho inteiro nos sistemas de computação. Elas podem ser de 8 bits, 16 bits, 32 bits, 64 bits e assim por diante.

De modo geral, as seguintes formas primitivas de dados são mais utilizadas nos programas atuais de computador (formas primitivas, entendidas pelo *hardware*):

- dados sob forma de caracteres (tipo caractere);
- dados sob forma lógica (tipo lógico);
- dados sob forma numérica (tipo numérico).

Outras formas mais complexas, como tipo registro, tipo conjunto, tipo estrutura e tipo ponteiro, podem ser utilizadas por linguagens de mais alto nível. No entanto, durante a compilação (geração do código), os dados acabam sendo convertidos nas formas primitivas anteriormente mencionadas.

Tipo caractere

A representação interna da informação, no *hardware* do computador, é realizada pela correspondência entre um símbolo da informação e um grupo de algarismos binários (bits). Isso é fácil de entender porque o computador (que é uma máquina binária) só possui dois símbolos (0 e 1) para sua representação e requer mais de 1 bit para identificar todos os símbolos que constituem as informações usadas pelo homem, que precisam ser armazenadas e processadas pela máquina.

Para representar todos os caracteres alfabéticos (maiúsculos e minúsculos), algarismos decimais, sinais de pontuação e de operações matemáticas etc., é necessário mais de 1 bit, ou seja, um grupo de bits que identifica univocamente o referido símbolo (caractere).

Na história do computador, várias codificações foram feitas, incluindo:

- *Binary Code Decimal* (BCD): grupo de 6 bits/caractere, permitindo a codificação de 64 caracteres (praticamente não é mais empregada).

- *Extend Binary Coded Decimal Interchange Code* (EBCDIC): grupo de 8 bits, exclusivo da IBM, permitindo a codificação de 256 símbolos diferentes.
- *American Standard Code for Information Interchange* (ASCII): usada pelos demais fabricantes.

Dentre os principais exemplos, temos o tipo *string*.

Tipo lógico

São dados que possuem apenas dois estados: verdadeiro ou falso. No computador, 0 representa falso e 1, verdadeiro.

Tipo numérico

Como o computador é uma máquina binária, faz melhor os cálculos em binário. A Unidade Lógica Aritmética (ULA) executa as operações mais rapidamente em binário.

Os subtipos inteiro e real são exemplos do tipo numérico.

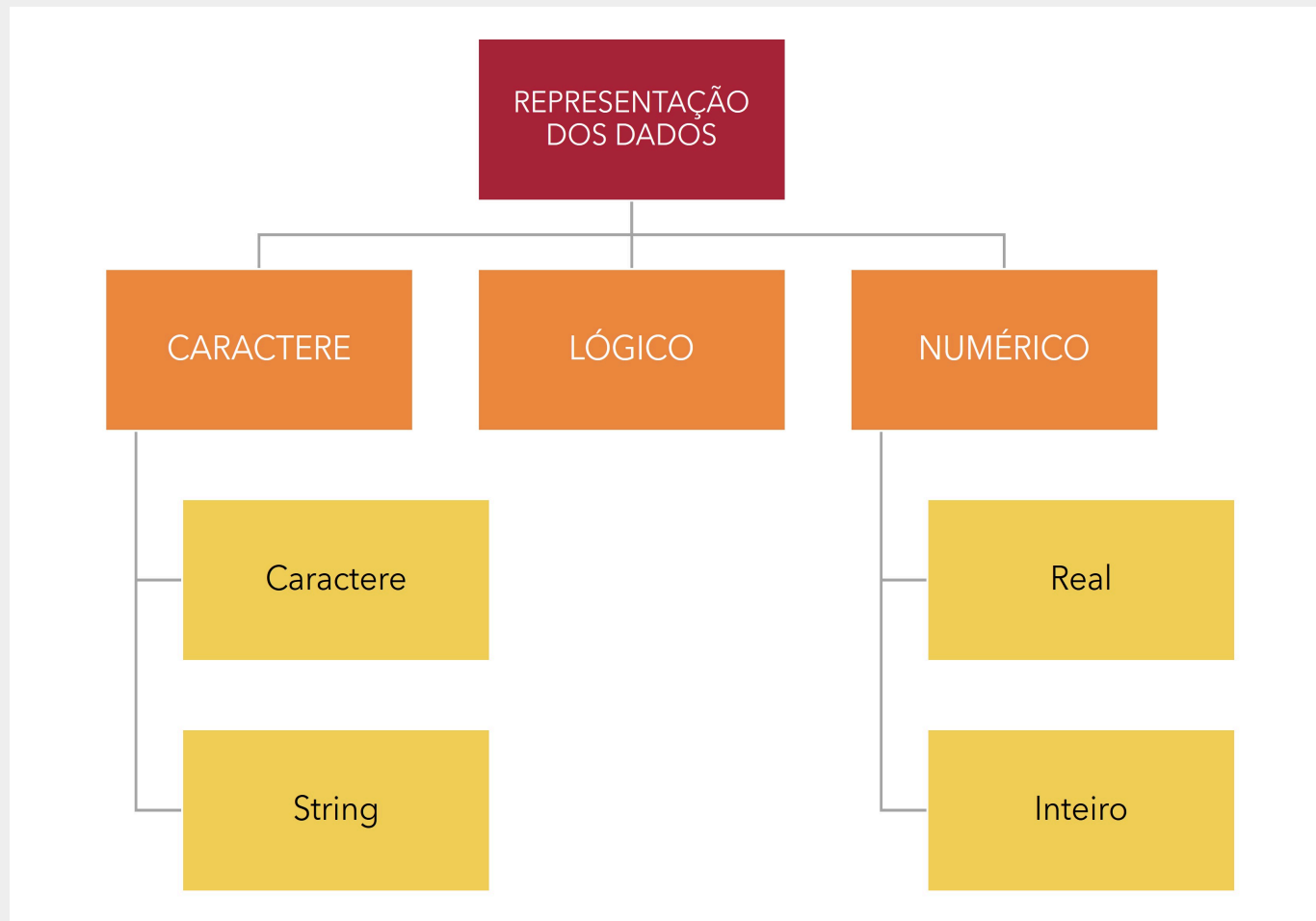


Figura 1: Representação dos dados. Fonte: O autor (2023).



SAIBA MAIS

Como o computador trabalha com números

Este vídeo ilustra como os números foram formados, sua importância e sua aplicação direta nos sistemas

computacionais. Também apresenta o funcionamento do hodômetro e sua aplicação direta nos sistemas computacionais.

Como o computador trabalha com números

PUCPR Online



Assista no

O computador e as linguagens de programação

Neste vídeo, são apresentadas as relações entre os sistemas digitais e o sistema de numeração binário, além de objetos da arquitetura de computadores.

O computador e as linguagens de programação

PUCPR Online



Assista no

| Conclusão

Nesta unidade, abordamos os temas necessários para o desenvolvimento dos sistemas de numeração, com ênfase no sistema binário. Os temas vistos foram:

sistemas de numeração não posicional;

- base;
- conversão de base;

- aritmética binária;
- adição binária;
- subtração binária;
- representação dos dados.

Ao final, fomos capazes de identificar como os computadores representam os dados internamente, converter números do sistema de numeração decimal para o sistema de numeração binário e realizar operações aritméticas básicas no sistema de numeração binário.

| Referências Bibliográficas

MARÇULA, M.; BENINI FILHO, P. A. Informática: conceitos e aplicações. 5. ed. São Paulo: Saraiva, 2009.

